Parallel Near-Duplicate Document Search with MinHash and LSH in Python

Yassin Belmaati
yassin.belmaati@edu.unifi.it
Course Project
Università degli Studi di Firenze

Abstract

This report presents the design and implementation of a parallel near-duplicate document search based on MinHashing and Locality-Sensitive Hashing (LSH). Starting from a sequential baseline, several parallel implementations are developed entirely in Python: SIMD vectorisation (NumPy), shared-memory multithreading with a released GIL (Numba `prange`), process-level data parallelism (joblib), I/O concurrency (asyncio) and GPU parallelism (Numba CUDA). The project explores thread scaling, scheduling and chunk sizing, memory-layout and vectorisation effects, synchronisation strategies for the LSH build, and GPU block-size tuning with global versus shared memory. All parallel versions are validated against the sequential baseline and benchmarked on a synthetic corpus with injected near-duplicates. The compute core (MinHash signatures) reaches a $649\times$ *speedup over a pure-Python baseline on a 16-thread CPU and a further* $6.8\times$ *on an entry-level GPU.*

1. Introduction

Detecting pairs of similar documents in a large collection is a fundamental primitive in data mining, appearing in plagiarism detection, web-crawl de-duplication, and clustering. Given a collection of D documents, the naive approach compares all $D(D-1)/2$ pairs, which is quadratic and infeasible for large D. Two classical ideas make the problem tractable.

First, each document is represented as the set of its k-shingles (length-k token sequences), and document similarity is measured by the *Jaccard similarity*

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}. \tag{1}$$

MinHashing compresses each set into a short integer *signature* of length N such that, for a hash function h_i drawn from a suitable family,

$$\Pr\left[\,\text{minhash}_i(A) = \text{minhash}_i(B)\,\right] = J(A,B). \tag{2}$$

The fraction of agreeing signature rows is therefore an unbiased estimator of $J(A,B)$. Second, **Locality-Sensitive Hashing (LSH)** splits the N-row signature matrix into b bands of r rows ($N = b\,r$) and hashes each document's per-band slice into a bucket; documents that collide in *any* band become *candidate* pairs. The probability that a pair of similarity s becomes a candidate is

$$1 - (1 - s^{\,r})^b, \tag{3}$$

an S-curve whose steep transition around $s \approx (1/b)^{1/r}$ is tuned to the desired threshold, so that only a tiny fraction of all pairs is ever compared.

The dominant cost of the pipeline is building the signature matrix, which is $O(\text{total\_shingles} \cdot N)$ and *independent per document*. This embarrassingly parallel structure is the target of the parallelisation. Python's Global Interpreter Lock (GIL) prevents plain threads from running CPU-bound bytecode in parallel, so the study deliberately spans the tools that circumvent it in different ways: NumPy (vectorised C loops), Numba (JIT-compiled native code with a released GIL), joblib (separate processes), asyncio (I/O concurrency) and Numba CUDA (GPU).

Section 2 describes the shared infrastructure. Section 3 presents the sequential baseline. Section 4 details the CPU parallel implementations and Section 5 the GPU one. Section 6 reports the experimental results.

2. Data Structures and Shared Infrastructure

All implementations share a common module (`core.py`) that defines the data representation, the reference algorithms, the dataset providers and the benchmark harness, so that every variant is measured and validated identically.

Core types. A corpus is stored in a CSR-like `ShingleDB`: the unique shingle ids of every document are concatenated into one flat `int64` array `vals`, with a

`offsets` array delimiting each document. This structure-of-arrays layout keeps the MinHash kernel a tight numeric loop and avoids one Python object per document. A `MinHashParams` holds the universal-hash coefficients a_i, b_i.

Numeric field. Shingles are hashed with a 64-bit polynomial rolling hash and reduced into the field $\mathbb{Z}_P$ with $P = 2^{31} - 1$, a Mersenne prime. Keeping shingle ids in $[0, P)$ guarantees that the universal-hash product $a_i x$ stays below 2^{62} and fits in a signed 64-bit integer, giving Numba a clean integer fast path with no overflow handling.

MinHash family. Each of the N hash functions is $h_i(x) = (a_i x + b_i) \bmod P$, with $a_i \in [1, P)$, $b_i \in [0, P)$. This family satisfies the estimator identity (2).

Benchmark harness. A generic `benchmark()` function times any callable, discarding two warm-up runs and then measuring N_{runs} times. It records both *wall-clock* time (`perf_counter`) and *CPU* time (`process_time`, summed over threads) and reports mean, standard deviation, min and max. The same harness is reused for every implementation.

Dataset. Synthetic corpora are generated with a Zipfian vocabulary distribution, which yields realistic shingle-overlap statistics. `n_dup_clusters` clusters of near-duplicates are injected by copying a base document and mutating a small fraction of its tokens; the within-cluster pairs constitute the *ground truth* for recall and precision. The generator scales freely, which is exploited for strong and weak scaling and to push the sequential baseline past ten seconds. Shingling (preprocessing) is timed separately and excluded from the MinHash measurements.

Algorithm 1 Sequential MinHash signatures

Require: CSR `vals`, `offsets`; coefficients a, b; prime P; hashes N
Ensure: signature matrix $\text{sig}[N][D]$

for $d \leftarrow 0$ to $D - 1$ **do**
  for $i \leftarrow 0$ to $N - 1$ **do**
    $m \leftarrow P$
    for $x \in$ shingles of document d **do**
      $v \leftarrow (a_i \cdot x + b_i) \bmod P$
      if $v < m$ **then** $m \leftarrow v$
      end if
    end for
    $\text{sig}[i][d] \leftarrow m$
  end for
end for

Algorithm 2 Numba parallel MinHash (`prange` over documents)

Require: CSR arrays, a, b, P, N
Ensure: $\text{sig}[D][N]$

for $d \leftarrow 0$ to $D - 1$ **in parallel** (`prange`) **do**
  initialise $\text{sig}[d][\,\cdot\,] \leftarrow P$
  for $x \in$ shingles of document d **do**
    for $i \leftarrow 0$ to $N - 1$ **do** ▷ unit-stride, vectorisable
      $v \leftarrow (a_i x + b_i) \bmod P$
      if $v < \text{sig}[d][i]$ **then** $\text{sig}[d][i] \leftarrow v$
      end if
    end for
  end for
end for

3. Sequential Algorithm

The sequential implementation is the correctness reference and performance baseline. It is provided in two forms: a pure-Python triple loop (which isolates interpreter overhead) and a NumPy-vectorised version (the honest sequential baseline). Both produce byte-identical signatures. Algorithm 1 summarises the MinHash core; LSH banding and verification follow directly from Section 2.

Candidate pairs are produced by hashing each document's per-band slice into a dictionary of buckets and emitting all within-bucket pairs; verification keeps the candidates whose estimated similarity (fraction of agreeing rows) is at least the threshold.

4. Parallel CPU Implementations

4.1. NumPy vectorisation

The first level of parallelism is data-level (SIMD). For a document with shingle vector X, the whole set of N hashes is evaluated at once as $(a\,X^\top + b) \bmod P$ followed by a row-wise minimum, delegating the inner loops to NumPy's vectorised C ufuncs. This removes interpreter overhead and uses the CPU's SIMD units without any explicit threading.

4.2. Numba: multithreading with a released GIL

The main CPU parallel implementation compiles the kernel with Numba's `@njit` and distributes the documents across threads with `prange`. Numba's parallel loops release the GIL and use the OpenMP threading layer, so the threads run genuinely in parallel; the number of threads is set with `numba.set_num_threads`. Algorithm 2 shows the kernel. The inner loop runs over the hash index i, so a_i, b_i and the signature entry are unit-stride, which is both cache- and SIMD-friendly. Two memory layouts are compared: *docs-major* (D, N), where each document's signature is a contiguous row, and *hashes-major* (N, D), where the per-hash writes are strided; `fastmath` is toggled as well.

Algorithm 3 LSH candidate generation (reduction)

Require: signature sig, bands b, threads p
Ensure: candidate pair set C

for thread $t \in \{0, \ldots, p-1\}$ **in parallel do**
  $C_t \leftarrow \emptyset$
  for each band assigned to t **do**
    hash each document's band slice to a bucket
    add all within-bucket pairs to C_t
  end for
end for
$C \leftarrow \bigcup_t C_t$ ▷ single-threaded union

4.3. joblib: process-level data parallelism

Where Numba parallelises inside one compiled function, joblib parallelises at the Python level by splitting the corpus into chunks of documents and running a worker per chunk. With the `loky` backend each worker is a separate process, side-stepping the GIL at the cost of pickling its inputs; each worker therefore receives only the slice of the CSR arrays it needs. The number of chunks is the scheduling knob, analogous to an OpenMP chunk size: too few chunks give load imbalance and no parallelism, too many are dominated by dispatch and serialisation overhead. A `threading` backend is kept as a contrast that exposes the GIL limitation.

4.4. asyncio: I/O-bound preprocessing

asyncio provides concurrency, not CPU parallelism: a single event loop interleaves tasks that *wait*. It is therefore the right tool for the document-loading stage (I/O-bound) and the wrong tool for MinHash (CPU-bound). The project measures both halves of this statement (Section 6), using `asyncio.gather` to overlap file-read latency and demonstrating that the same mechanism gives no speedup on CPU work.

4.5. Synchronisation: the LSH build

Building the candidate set is a reduction into a shared associative structure, which is the Python analogue of the OpenMP "critical versus reduction" question. Three strategies are implemented and compared (Algorithm 3 shows the reduction): (i) `reduction`, where each thread hashes a disjoint set of bands into a private candidate set and the sets are unioned once at the end; (ii) `shared_lock`, a single lock taken on every insert into one shared set (fine-grained); and (iii) `coarse_lock`, one lock acquisition per thread to merge a local set (coarse-grained).

5. GPU Implementation

The GPU implementation uses Numba's CUDA target, keeping the code in Python and consistent with the rest of the project. It exploits the same per-document data parallelism at a much finer granularity. Two kernels are provided.

Algorithm 4 CUDA kernel: one thread per (document, hash)

$t \leftarrow \text{blockIdx}.x \cdot \text{blockDim}.x + \text{threadIdx}.x$
if $t \geq D \cdot N$ **then return**
end if
$d \leftarrow t/N; \quad i \leftarrow t \bmod N$
$m \leftarrow P$
for $x \in$ shingles of document d (global memory) **do**
  $v \leftarrow (a_i x + b_i) \bmod P;$
  if $v < m$ **then** $m \leftarrow v$
  end if
end for
$\text{sig}[d][i] \leftarrow m$

Global-memory kernel. One thread per $(\text{document}, \text{hash})$ pair, flattened onto a 1-D grid so that the launch's threads-per-block is a single sweepable parameter (Algorithm 4). Adjacent threads handle consecutive hash indices of the same document, so reads of a_i, b_i and writes of the signature are coalesced.

Shared-memory kernel. One block per document, N threads per block. The block cooperatively stages the document's shingles into shared memory a tile at a time; every thread (one per hash) then reuses the tile, eliminating the redundant global loads of the shingle stream that the global kernel incurs.

Memory management. Device buffers are allocated once and reused across runs; the host-to-device transfer of the corpus is therefore excluded from the measured time, which covers kernel execution and the copy of the resulting signature back to the host. The block size is swept over $\{32, 64, 128, 256, 512, 1024\}$.

6. Tests and Results

6.1. Experimental setup

Hardware. 11th Gen Intel Core i7-11800H (8 physical cores, 16 logical threads); NVIDIA T1200 Laptop GPU
Software. Ubuntu 24.04, Python 3.12, NumPy, Numba, joblib.
Dataset. Synthetic corpus: $D = 6000$ documents, vocabulary 30000 (Zipfian), mean length 400 tokens, shingle size $k = 5$, $N = 256$ hashes, 2.38×10^6 total shingles, 60 injected near-duplicate clusters.
Methodology. Each configuration is executed 10 times after two discarded warm-up runs (Numba kernels are additionally JIT-warmed); the reported metric is the mean wall-clock time. Only computation is measured; I/O and allocation are excluded.

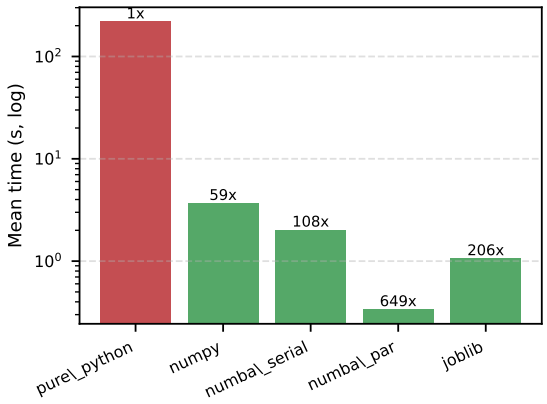

Figure 1. MinHash implementation comparison (log scale, $D = 6000$, $N = 256$). Annotations are speedup over pure Python.

Implementation	Mean (s)	Speedup
pure Python	218.79	1.0×
NumPy vectorised	3.680	59×
Numba serial	2.021	108×
joblib (loky)	1.061	206×
Numba parallel 16t	0.337	649×

Table 1. Implementation comparison, speedup relative to pure Python.

6.2. Correctness validation

Correctness is validated by three independent checks. (i) The MinHash estimator is unbiased: the mean absolute error between the estimated and the true Jaccard similarity is 0.025. (ii) The pure-Python and NumPy signatures are identical, and *every* parallel variant (Numba, joblib, both CUDA kernels) is asserted byte-for-byte equal to the reference before it is timed, which guarantees race-freedom. (iii) End-to-end, LSH recovers the injected near-duplicates with recall 1.00 and precision 1.00, examining only 75 candidate pairs versus 319,600 for brute force, a 99.98% reduction. The CUDA kernels were validated on-device and, without a GPU, via the Numba CUDA simulator.

6.3. Implementation comparison and vectorisation

Figure 1 and Table 1 compare the implementations on the full corpus. Two independent sources of speedup are visible. Removing interpreter overhead and using SIMD (NumPy) already gives 59× over pure Python, and Numba's native code 108×, *before any multicore parallelism*. Adding 16-thread parallelism (Numba) reaches 649×. The isolated vectorisation experiment (Figure 2) confirms the same effect on a fixed subset: NumPy is 64× and Numba 103× faster than the pure-Python loop.

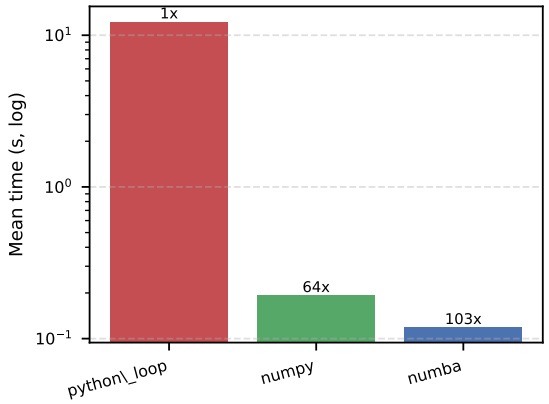

Figure 2. Vectorisation: pure-Python loop vs NumPy vs Numba on a fixed subset.

Threads	Mean (s)	Speedup	Efficiency
1	1.997	1.00×	1.00
2	1.028	1.94×	0.97
4	0.677	2.95×	0.74
8	0.455	4.39×	0.55
16	0.310	6.45×	0.40

Table 2. Numba thread scaling (strong scaling, $D = 6000$, $N = 256$).

6.4. Strong scaling

Table 2 and Figure 3 report the Numba speedup against thread count. Scaling is near-ideal up to 2 threads and remains good to 8 physical cores (4.39×, 55% efficiency), but the step from 8 to 16 threads adds only 1.47×: the extra logical threads are hyperthreads sharing physical execution units, not independent cores. The efficiency curve (Figure 4) falls from 0.97 to 0.40. The gap from ideal is consistent with Amdahl's law and with the memory-bound nature of the kernel: the inner loop performs little arithmetic per shingle read, so aggregate memory bandwidth, rather than compute, limits scaling.

6.5. Weak scaling

In the weak-scaling experiment the problem size grows proportionally with the thread count (375 documents per thread), so the ideal is a constant time (efficiency 1.0). Figure 5 shows the efficiency declining to 0.38 at 16 threads, mirroring the strong-scaling result and confirming that the same memory-bandwidth and hyperthreading effects, not a fixed serial section alone, dominate at high thread counts.

6.6. Memory layout and vectorisation variants

Figure 6 compares the docs-major and hashes-major layouts and the `fastmath` toggle. The three variants are statistically indistinguishable (all $\approx$ 0.31 s, within one stan-

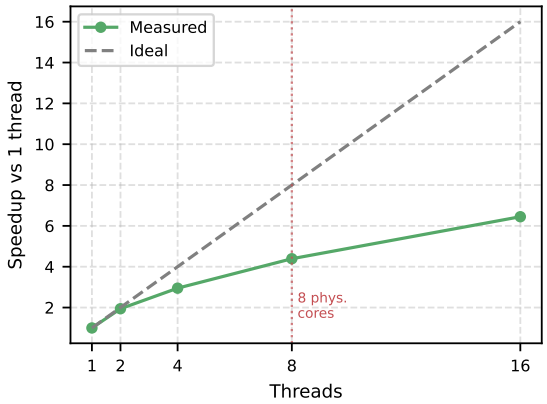

Figure 3. Numba strong scaling: speedup vs ideal linear scaling. The dotted line marks the 8 physical cores.

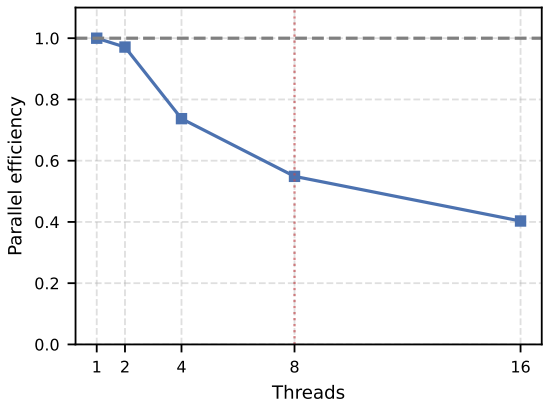

Figure 4. Parallel efficiency. The knee at 8 threads reflects the transition from physical cores to hyperthreads.

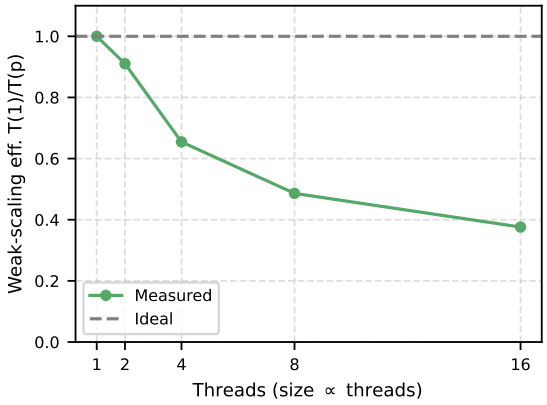

Figure 5. Weak scaling: problem size grows with the thread count.

dard deviation). At $N = 256$ a signature row is only 2 KB and fits in L1, and the kernel is dominated by the modulo operation rather than by memory traffic, so the layout penalty is negligible at this scale; `fastmath` has no effect

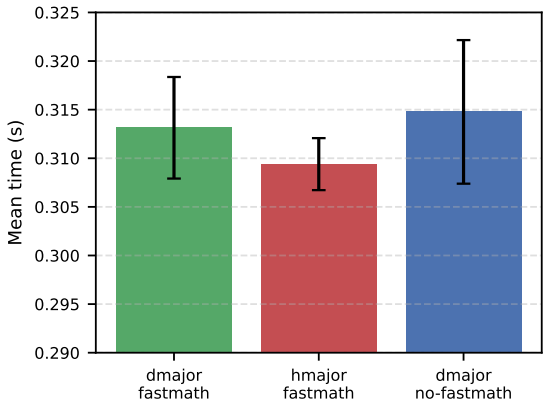

Figure 6. Signature layout and `fastmath` variants (error bars: std). The differences are within noise at $N = 256$.

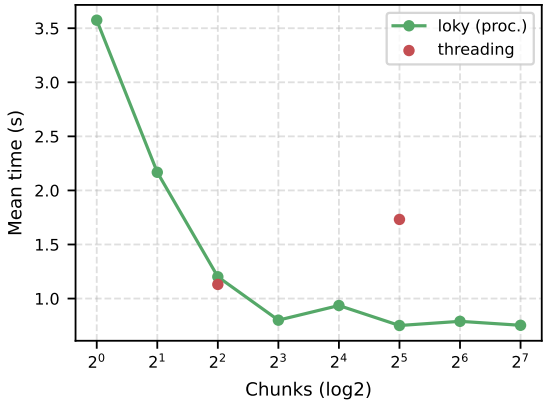

Figure 7. joblib: mean time vs number of chunks ($\log_2$), loky vs threading.

because the kernel arithmetic is integer.

6.7. Scheduling: joblib chunk size and backend

Figure 7 shows the joblib time against the number of chunks. With a single chunk the work is serial (3.57 s); the time drops sharply and reaches a plateau around 0.75–0.80 s for 8 or more chunks, after which additional chunks only add dispatch and pickling overhead. The best configurations split the work into more tasks than physical cores, which improves load balance across the 8 cores. The `threading` backend is markedly slower (e.g. 1.73 s at 32 chunks) because the GIL serialises the Python-level dispatch, confirming why processes are required for CPU-bound Python work.

6.8. Synchronisation: the LSH build

Figure 8 compares the three synchronisation strategies. None of them speeds up with more threads: the bucket build is dominated by Python dictionary/set operations, which

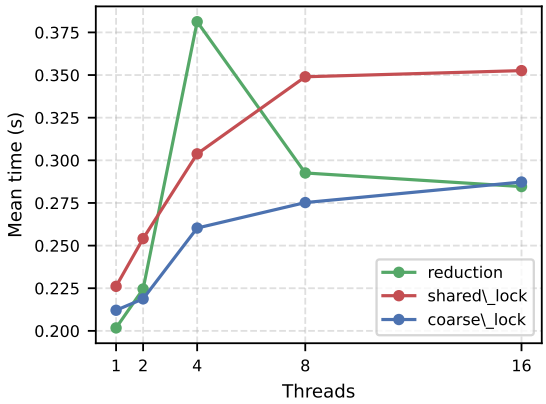

Figure 8. LSH build: fine-grained locking vs coarse locking vs reduction.

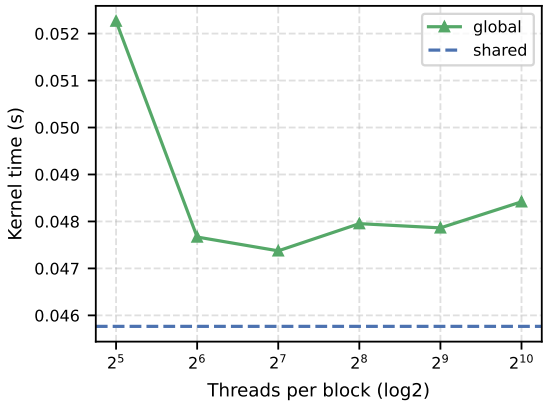

Figure 9. CUDA block-size sweep (global kernel) with the shared kernel as reference.

the GIL serialises. The informative signal is relative: the fine-grained `shared_lock` degrades monotonically with thread count ($0.23\,\mathrm{s} \rightarrow 0.35\,\mathrm{s}$) as lock contention grows, whereas the `reduction` and `coarse_lock` strategies, which touch the shared structure at most once per thread, remain lower and flatter.

6.9. GPU

Figure 9 shows the CUDA block-size sweep. Block size 32 is the slowest ($0.0523\,\mathrm{s}$) because small blocks underutilise the GPU's warp scheduler; from 64 upward the time plateaus around 0.047–$0.048\,\mathrm{s}$. The shared-memory kernel is the fastest ($0.0458\,\mathrm{s}$), improving on the best global configuration by staging the reused shingle stream in on-chip memory. Table 3 and Figure 10 compare the GPU against the already-parallel 16-thread CPU: the shared kernel is $6.85\times$ faster.

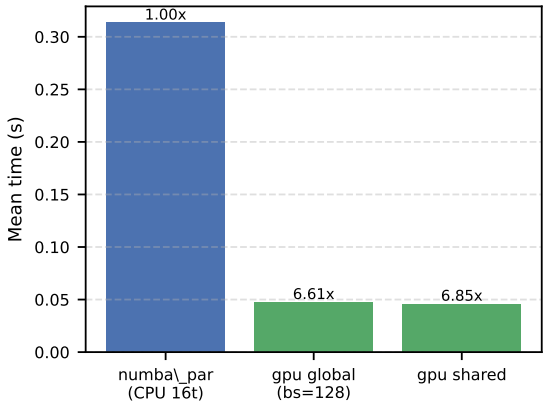

Figure 10. GPU vs the 16-thread CPU baseline. Annotations are speedup over the CPU.

Configuration	Mean (s)	Speedup vs CPU
Numba parallel (CPU, 16t)	0.3133	1.00×
CUDA global (block 128)	0.0474	6.61×
CUDA shared	0.0458	6.85×

Table 3. GPU vs CPU MinHash (kernel time, transfer amortised).

6.10. Discussion

The results separate cleanly into two axes of speedup. The first, and larger single jump, is removing interpreter overhead and enabling SIMD (NumPy, Numba serial): two orders of magnitude with no multicore involved. The second is multi-core and GPU parallelism on top. Numba threads share memory and release the GIL, so they scale without serialisation cost but are ultimately limited by memory bandwidth and by hyperthreading beyond 8 cores; joblib processes achieve comparable parallelism but pay a pickling cost that chunk sizing must amortise. asyncio, by contrast, gives no CPU speedup at all, which is the expected and instructive result. The GPU adds a further order of magnitude over the parallel CPU for a large enough problem, once the launch and transfer latency is hidden.